
Compressing Chain-of-Thought Length in Small Language Models via GRPO

Filip Łazowski
University of Warsaw
f1448341@students.mimuw.edu.pl

Bartłomiej Cupiał*
University of Warsaw
b.cupial@uw.edu.pl

Abstract

This work presents a reinforcement learning approach to compress Chain-of-Thought reasoning in small language models without degrading accuracy. Shortening reasoning through prompt engineering lowers mathematical performance, whereas enforcing brevity via a naive length penalty in Group Relative Policy Optimization triggers reward hacking and policy collapse. To resolve this, the training objective is regularized against the base model to ensure controlled compression. Evaluated on the GSM8K dataset with the Qwen2.5-0.5B model, the optimized policy achieves 49 percent accuracy at a mean length of 222 tokens. This improves accuracy by 9 percentage points while reducing reasoning length by 23 percent compared to the untrained baseline.

1 Introduction

Language models use Chain-of-Thought reasoning to decompose complex problems into simpler, smaller tasks [1, 2]. A common intuition in the field has been that longer and more detailed reasoning steps lead to better performance, particularly for mathematically difficult tasks [3, 4]. However, recent research demonstrates that reasoning performance does not consistently improve as chains grow longer, and instead, task accuracy typically follows an inverted U-shaped curve with respect to CoT length [5]. While accuracy initially improves as the task is appropriately decomposed, it eventually degrades in longer chains due to error accumulation [5].

To optimize CoT length in small models, the Group Relative Policy Optimization [7] was employed. Unlike capable models that naturally reduce reasoning length during standard reinforcement learning [5], enforcing brevity on small models via a length penalizing reward in GRPO introduces optimization barriers. Specifically, it triggers reward hacking mechanism [6], where the policy collapses by bypassing reasoning entirely to maximize the brevity reward. To address this issue and investigate the trade-off between CoT length and accuracy, this work proposes a two-phase framework: first establishing a reasoning baseline through prompt engineering, and second stabilization of length penalized GRPO.

1.1 Experimental Setup

The model selected for all experiments is Qwen2.5-0.5B-Instruct [8], evaluated on the GSM8K dataset [9]. For the reinforcement learning stage, the training pipeline utilizes a modified TinyZero [10] framework, adapted to support custom reward logic and length penalization. All computations are executed on a RunPod instance with a single NVIDIA RTX 4090 GPU (24GB VRAM).

*Supervisor of the project

2 Phase 1: Empirical Analysis of CoT Length and Prompting Baseline

The first phase explores the relation between CoT length and accuracy for the base model, and introduces an untrained baseline for shortening reasoning through prompt engineering. All evaluations are performed on the first 200 problems of the GSM8K test split using greedy decoding. Because this model seems to disobey standard response formatting, correctness is scored with a custom extractor that parses different kinds of numeric responses - this avoids the potential false negatives.

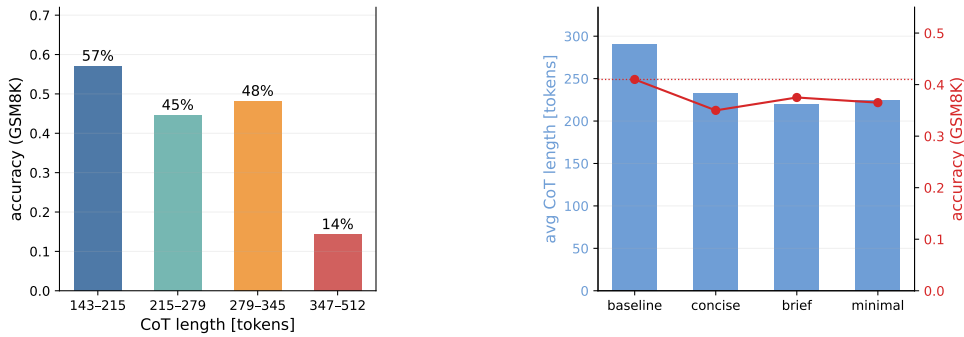
Natural baseline and the length-accuracy relationship. With empty system prompt, the model scores 41% accuracy with a mean CoT length of 290 tokens. Partitioning these answers into length quartiles shows a relationship between reasoning length and correctness (Figure 2a): the shortest quartile (143 - 215 tokens) has 57% of accuracy, whereas the longest quartile (347 - 512 tokens) collapses to 14%. This is consistent with the inverted-U behaviour¹ reported by [5]. For this model, length acts as a sign of difficulty, meaning that harder problems cause longer, error prone chains, rather than as a cause of improved performance.

Prompt engineering for brevity. Next experiment tested whether reasoning can be shortened by instruction alone. Three brevity oriented system prompts (*brief*, *concise*, *minimal* - Figure 1) are compared against the baseline (Figure 2b). Prompting does compress the chains, but at a accuracy cost. The *brief* prompt gives the best trade-off, reaching 37% accuracy at 220 tokens (a 24% length reduction for a 4 points accuracy drop), while the *concise* and *minimal* prompts compress to 233 and 224 tokens but drop the accuracy to 35% and 36.5% respectively.

"brief": "Solve the math problem. Think step by step,
but keep your reasoning brief."
"concise": "Solve the math problem using as few reasoning steps as possible.
Be concise, then give the final answer."
"minimal": "Solve the math problem. Give only the minimal reasoning needed,
then state the final answer."

Figure 1: Three system prompts used for the reasoning shortening.

Together, these results define the baseline that the reinforcement learning phase seeks to surpass. Prompt engineering shows that shorter reasoning is attainable only by sacrificing accuracy. Phase 2 investigates whether a length penalized GRPO objective can move beyond this, producing shorter chains at same or higher accuracy.



(a) Accuracy by CoT length quartile. The longest answers are the least accurate. **(b)** Brevity prompts shorten the CoT but lower accuracy. *Brief* is the best trade-off in the Phase 1.

Figure 2: Phase 1 on Qwen2.5-0.5B-Instruct / GSM8K.

¹Note that pure inverted-U would show if the model wasn't tuned for reasoning, as of now it is not capable of producing straightforward, short answers

3 Phase 2: Compressing Reasoning with Length Regularized GRPO

In Phase 2 a training to shorten the model’s reasoning with GRPO occurs. In one training step a batch of 64 prompts is being sampled and, for each prompt, $G = 8$ completions (o_i) is generated with temperature sampling ($T = 0.7$) from the policy π_{old} that generates this step’s completions. Temperature sampling is used so the completions in one group differ rather than give the same answer. Each completion o_i of length ℓ_i tokens is scored by a reward that combines correctness with a length penalty,

$$R_i = \mathbf{1}[\text{correct}_i] - \lambda \frac{\ell_i}{512},$$

where $\mathbf{1}[\text{correct}_i]$ is 1 when the answer in o_i is correct and 0 otherwise. The length is normalized by a constant so that λ means the same thing across runs. Because GRPO does not use a critic, the group itself acts as one - within each group of 8 the reward produce the advantage $\hat{A}_i = (R_i - \text{mean}_g)/(\text{std}_g + \varepsilon)$ and assign it to every token of o_i . The group of size 8 was chosen to provide stable signal and reducing computing cost (on single GPU), and also to produce diverse completions - if all completions are all wrong (or all correct) λ terms cancels out in the group’s advantage and its value doesn’t matter in this group. Then a single gradient update on the batch is done (one minibatch, one epoch) that maximizes the GRPO objective of DeepSeek [7],

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{q \sim P(Q), \{o_i\} \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} L_{i,t} \right],$$

$$L_{i,t} = \min(\rho_{i,t} \hat{A}_{i,t}, \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon) \hat{A}_{i,t}) - \beta \mathbb{D}_{\text{KL}}[\pi_{\theta} \parallel \pi_{\text{ref}}].$$

Here $\rho_{i,t} = \pi_{\theta}(o_{i,t} \mid q, o_{i,<t})/\pi_{\theta_{\text{old}}}(\cdot)$ is the importance ratio, $\hat{A}_{i,t} = \hat{A}_i$ is advantage for every token t of o_i (the reward is a single scalar per answer), ϵ is the clip ratio, and $\beta \geq 0$ the KL coefficient. Since exactly one gradient update is run per rollout, $\rho_{i,t} = 1$: the clip is never used and one training step simply is a group relative policy gradient with a KL penalty - $\mathbb{D}_{\text{KL}}$ (Schulman’s low-variance k3 estimator [11]).

A sign issue. The KL term above is meant to penalize divergence from the base, with $\beta \geq 0$. It was found that the TinyZero implementation that was used combines this term into the loss with the wrong sign, so that a positive `kl_loss_coef` ends up rewarding divergence rather than penalizing it, which was the default for the GRPO. It was verified in the implementation and directly on the $\lambda = 0$ with a positive coefficient (+5) which drives the policy away from the π_{ref} and collapses accuracy to 13.9%. Whereas with a negative coefficient (−5) KL is near zero with accuracy 42.5%.

All accuracy and length numbers below use the same greedy-decoded GSM8K test split as Phase 1, but with the training prompt, which append system prompt to reason step by step and give the final answer after a ####. With this prompt the untrained base model scores 40.0% at a mean of 287 tokens (median 265), one point below the 41% it scores with the empty prompt in Phase 1.

The reward hacking. First runs trained for 50 steps. With any positive λ this leads to the model not learning to reason more briefly, it learns to skip reasoning instead. It finds that generating a random number in the response collects the length reward, and accuracy drops almost to zero - and the more accuracy drops the faster it degrades - see λ term cancelation above).

Figure 3 shows this on the 50-step run: the mean response length drops from about 350 tokens to about 5, while the KL from the base model rises from 0 to about 7 (specifically it’s `kl_loss` metrics). This is known as reward hacking, and in our case the length term is optimized at the cost of the accuracy. Note that the KL is measured against the reference, so its spike happens together with the length compression and partly measures how far the policy has moved, not correctness on its own. It still acts as a good metrics for testing whether model has degraded too much and differs from the base model, which one would like to avoid to some extent.

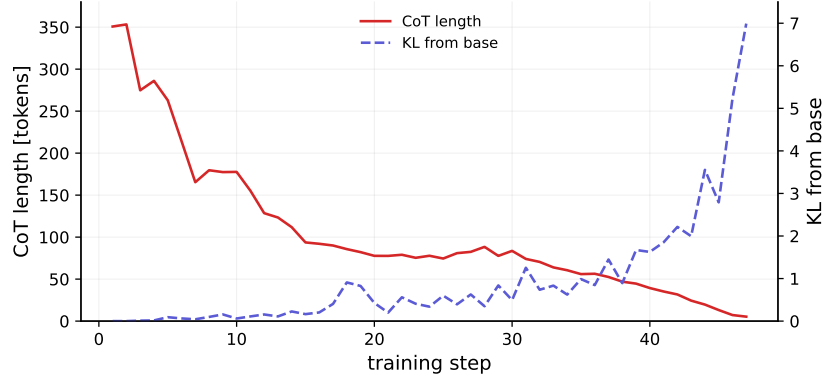


Figure 3: Reward hacking on the 50 steps run ($\lambda = 0.1$, default β): the mean CoT length collapses to a few tokens while the KL from the base explodes to about 7, meaning that the policy stops reasoning to collect the length reward.

Controlled compression. The fix for reward hacking has two parts. Firstly, setting correct KL penalty (see 3), which keeps the policy close to a base model so it shortens the CoT in a controlled way instead of collapsing - this part will be referred as KL anchor. Second part is to lower the training loop steps, because even with the KL anchor the policy eventually degrades too much. It is set specifically to 15 steps, which has proved to work the best during test runs. Figure 4 shows the training dynamics for $\lambda \in \{0, 0.01, 0.05, 0.1\}$ at $\beta = 0.01$. The most important conclusion from these graphs is that the CoT length drops steadily for positive lambdas, while the KL does not increase too much compared to former runs without KL anchor. Reward is not that important to use as a metrics, as it depends strongly on the noise caused by sampling - what’s more important is that the model doesn’t degrade too much so the reward for responses could still have a positive impact on model’s policy update. Note that for $\lambda = 0$ length stays at the base and its KL barely change, so the compression and the KL climb in the other runs are driven by the length reward and that no signal collapses, in contrast to a length penalty without the anchor (Figure 3).

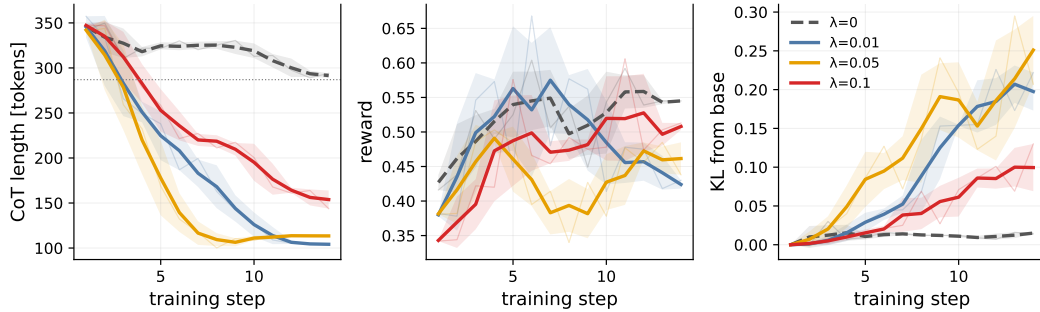
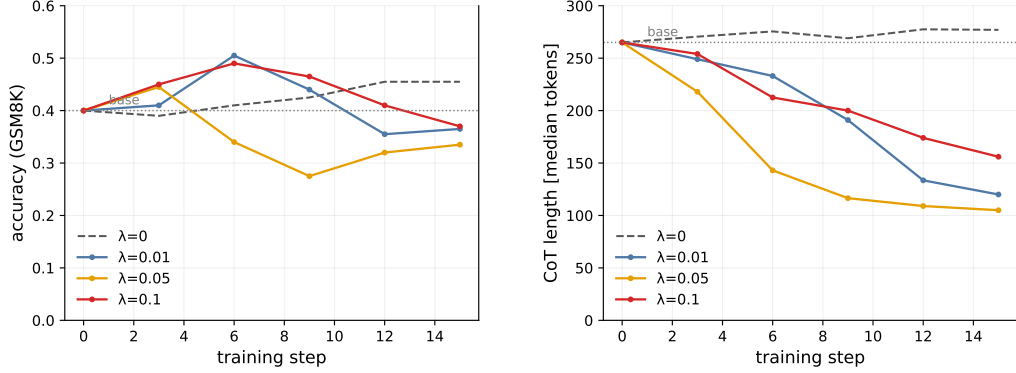


Figure 4: Chosen metrics during GRPO training for each λ at $\beta = 0.01$ ($\text{kl_loss_coef} = -0.01$). Left: response length (dotted line is the Phase 1 baseline). Middle: reward. Right: KL from base. Single run noise - solid line is a rolling mean and band is rolling std.

3.1 Evaluation and trade-offs

Because the KL anchor tries to keep the model at the base while the length reward pulls the other way, the two need to be balanced. In Figure 5 every λ first increases above the base model, peaking around 45 to 50% at steps 3 to 6, then later accuracy is traded for more CoT compression. Both $\lambda = 0.01$ and $\lambda = 0.1$ keep accuracy above the base while compressing, $\lambda = 0.05$ compresses fastest but overshoots, dropping below the base by step 6.



(a) Accuracy vs. step. The $\lambda > 0$ runs rise above the base (40%) at the beginning. The $\lambda = 0$ keeps around the baseline and never compresses. (b) Median CoT length vs. step. The $\lambda = 0$ stays at the base, while $\lambda > 0$ runs compress the CoT.

Figure 5: Evaluation of length penalized GRPO with a KL penalty on GSM8K test split ($\beta = 0.01$, $\text{kl_loss_coef} = -0.01$), $\lambda \in \{0.01, 0.05, 0.1\}$, 15 steps. The compression is controlled and trades off against accuracy, the useful checkpoints are early to mid training.

What matters. Note that because the training loop is shortened, the metrics can be noisy and the exact values should not be overinterpreted. Restriction to 15 steps causes each λ to not affect the compression rate accordingly. This is visible on the charts, where $\lambda = 0.1$ doesn’t compress much faster than smaller ones. At this scale, the choice of the λ is not that important, as long as $\lambda > 0$, which successfully guide the policy towards CoT shortening without losing much accuracy. Relation where a larger λ causes rapid compression, only shows up during longer trainings, which of course triggers reward hacking and model degradation. Because the goal was to find a practical trade-off, this work focused on finding the best stable configuration for this small model.

Proper comparison. Table 1 lists the best checkpoints. The highest accuracy for $\lambda = 0.1$ at step 6, reaches 49.0% at a mean of 222 tokens (9 points above the Phase 2 base and 23% shorter). Trading accuracy for length, $\lambda = 0.01$ at step 9 gives 44.0% at 192 tokens and $\lambda = 0.1$ at step 12 reaches 188 tokens while holding the Phase 2 base accuracy (40%). The contrast with Phase 1 shows that GRPO works better for compressing CoT without losing accuracy.

Table 1: Evaluation on GSM8K. Best chosen.

Phase	Configuration	Accuracy	Mean length
1: Prompting	Empty	41.0%	290
	<i>Brief</i> prompt	37%	220
2: GRPO	Untrained base	40.0%	287
	$\lambda = 0.1$ (step 6)	49.0%	222
	$\lambda = 0.01$ (step 9)	44.0%	192
	$\lambda = 0.1$ (step 12)	41.0%	188

Evaluation example. Figure 6 shows three different policies on one problem. The base model generate the solution with 223 tokens, with numbered steps and full sentences. The GRPO policy ($\lambda = 0.1$, step 9) reaches the same correct answer in 54 tokens keeping reasoning, but dropping the redundancy. The third policy without KL anchor generate the wrong number 11 with no reasoning, which is collapse that the KL anchor stops.

Problem (idx 91): Jan has three times the number of pets as Marcia. Marcia has two more pets than Cindy. If Cindy has four pets, how many total pets do the three have? (Correct answer: 28)

[base model, 223 tokens, correct]

To find out how many pets Jan, Marcia, and Cindy have in total, we need to follow these steps:

1. Determine how many pets Marcia has.
2. Use that information to determine how many pets Jan has.
3. Add up all the pets.

First, ... Marcia has $4 + 2 = 6$ pets.

Next, ... Jan has three times Marcia, so [...] (trimmed)

[GRPO: beta=0.01, lambda=0.1, step 9, 54 tokens, correct]

Marcia has $4 + 2 = 6$ pets. Jan has $6 * 3 = 18$ pets.

The three have a total of $18 + 6 + 4 = 28$ pets. 28.

[GRPO: reward hacked for lambda=0.1, 2 tokens, wrong]

11

Figure 6: Three policies on the same GSM8K problem.

4 Conclusion.

Three main things are concluded from this work. First, the KL penalty is effective for length regularized GRPO, preventing policy collapse caused by the length penalty alone. Second, the optimization itself is a strict trade-off between two opposing forces: the KL anchor resists compression by pulling the policy toward the baseline, while the length penalty degrades accuracy if pushed too far - the best policies are found in early to mid training, where the CoT is already shortened while accuracy remains at or above the base. Third, Phase 2 achieves what Phase 1 could not: prompting traded accuracy for brevity, whereas GRPO produces reasoning that is both shorter and maintains accuracy compared to the base model.

Findings and future work. To show that long CoTs are correlated to lower accuracy, negative λ training was performed, which rewards longer chain instead of penalizing it. This makes the chain to grow toward the 512-token limit while accuracy not only does not improve but it drops - Figure 7. This matches the Phase 1 (together with [5]) finding that length is a symptom of difficulty, not correctness (because of potential errors accumulation). On the reward design: reward hacking comes from the reward definition, in which a short wrong answer scores better than a long wrong answer. A more direct fix would be to apply the length penalty only to correct answers, $R = 1[\text{correct}] (1 - \lambda \ell / 512)$, which prevents the policy from being brief at the cost of correctness.

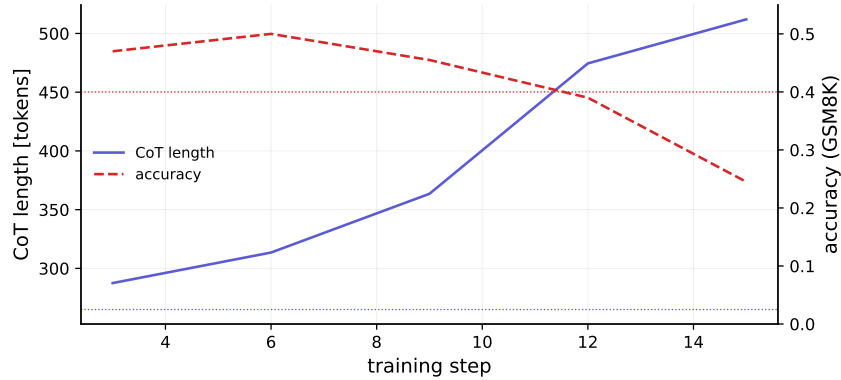


Figure 7: $\lambda = -0.05$. More doesn't mean better.

References

- [1] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, 2022. *arXiv:2201.11903*
- [2] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V Le, and Ed H. Chi. Least-to-Most Prompting Enables Complex Reasoning in Large Language Models, 2022, *arXiv:2205.10625*
- [3] Yao Fu, Hao Peng, Ashish Sabharwal, Peter Clark, and Tushar Khot. Complexity-based prompting for multi-step reasoning, 2023. *arXiv:2210.00720*
- [4] Mingyu Jin, Qinkai Yu, Dong Shu, Haiyan Zhao, Wenyue Hua, Yanda Meng, Yongfeng Zhang, and Mengnan Du. The Impact of Reasoning Step Length on Large Language Models, 2024. *arXiv:2401.04925*
- [5] Yuyang Wu, Yifei Wang, Ziyu Ye, Tianqi Du, Stefanie Jegelka, Yisen Wang. When More is Less: Understanding Chain-of-Thought Length in LLMs, 2025. *arXiv:2502.07266*
- [6] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krashenninikov, David Krueger. Defining and Characterizing Reward Hacking, 2022. *arXiv:2209.13085*
- [7] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y.K. Li, Y. Wu, Daya Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, 2024. *arXiv:2402.03300*
- [8] Qwen Team et al. Qwen2.5 Technical Report, 2024. *arXiv:2412.15115*
- [9] Cobbe, K., Kosaraju, V., Bavarian, M., et al. (2021). Training Verifiers to Solve Math Word Problems. *arXiv:2110.14168*.
- [10] Jiayi Pan. TinyZero: A clean, minimal, and accessible implementation of DeepSeek-R1-Zero, 2025. *GitHub repository*
- [11] John Schulman. Approximating KL Divergence . *Web article*

Coding assistant usage

A coding assistant (Claude) was used to generate the figures and to help write the RunPod launch scripts.